

ARCHITECTURE & DESIGN DOCUMENT

Dynamic Resilient Grid (DRG)

AI-driven local electricity demand forecasting and electrification stress analysis for UK urban networks



Document	DRG-SAD-001 Software Architecture & Design Document
Version	1.0 (Baseline)
Status	Issued for review
Date	28 August 2026
Classification	Internal - contains no personal or licensed smart-meter data
Companion	DRG-BRD-001 Business Requirements Document

1. Document Control

Table 1.1 - Revision history

Version	Date	Description	Author
0.1	2026-08-20	Initial draft: scope, context, component decomposition	Solution Architect
0.2	2026-08-25	UML model added; electrification simulation design reviewed and recalibrated	Solution Architect
0.3	2026-08-27	Azure deployment architecture; observability and governance sections	Cloud Architect
1.0	2026-08-28	MLOps promotion gate and blue/green model release added; baselined	Solution Architect

Table 1.2 - Approvals required

Role	Responsibility for this document	Signature
Project Sponsor	Confirms the architecture serves the business case in DRG-BRD-001	
Network Planning Lead (DNO)	Confirms the stress definition and scenario parameters are fit for planning use	
Data Protection Officer	Confirms compliance with the UK Data Service End User Licence	
Cloud Platform Owner	Confirms the Azure landing-zone design and identity model	
Academic Supervisor	Confirms methodological soundness against the project proposal	

1.1 Related documents

Table 1.3 - References

Reference	Title
DRG-BRD-001	Business Requirements Document (companion to this document)
REF-01	UK Data Service Study 7857 - Low Carbon London smart-meter trial, half-hourly consumption 2011-2014
REF-02	National Grid ESO Carbon Intensity API - <code>api.carbonintensity.org.uk</code>
REF-03	Open-Meteo ERA5 reanalysis archive and forecast API
REF-04	OpenWeatherMap Current Weather Data API
REF-05	Repository documentation: <code>README.md</code> , <code>docs/ARCHITECTURE.md</code> , <code>docs/AZURE.md</code>

1.2 Glossary of terms used in this document

Table 1.4 - Definitions and abbreviations

Term	Definition
ADMD	After Diversity Maximum Demand - the peak electrical demand attributable to one connected customer or device once fleet coincidence is accounted for. The standard planning quantity used by UK DNOs.
DNO	Distribution Network Operator - the licensed operator of a regional low- and medium-voltage electricity network.
Half-hour period	The settlement granularity of GB electricity data; 48 periods per day, indexed 0-47.
Heating degrees (HDD)	Degrees Celsius by which outside temperature falls below a balance-point temperature; the driver of electrified heating demand.
LV feeder	Low-voltage circuit supplying a cluster of domestic properties from a distribution substation.
Neighbourhood	In DRG, a fixed-size aggregation of smart-meter households used as the unit of analysis in place of a physical LV feeder, which the source data does not identify.
Stress period	A half-hour in which aggregated neighbourhood demand equals or exceeds the statistically defined threshold (P95 of that neighbourhood's own history).
Peak amplification	Percentage increase in the maximum half-hourly demand of a neighbourhood under an electrification scenario, relative to the un-electrified base case.
Champion model	The forecasting model currently selected for serving, chosen on held-out test performance.
Promotion gate	The automated quality decision that determines whether a retrained model may replace the deployed one.
SHAP	SHapley Additive exPlanations - a game-theoretic method attributing a model prediction to its input features.

2. Contents

1 Introduction

1.1 Purpose · 1.2 Intended audience · 1.3 How to read this document

2 Project Overview

2.1 Problem statement · 2.2 Aim · 2.3 Objectives · 2.4 Research questions · 2.5 What the system is not

3 Scope

3.1 In scope · 3.2 Out of scope · 3.3 Assumptions · 3.4 Constraints

4 Stakeholders and System Actors

5 System Context

6 Architecture Overview

6.1 Architectural drivers · 6.2 Layered view · 6.3 Design principles · 6.4 Key decisions

7 UML Model

7.1 Use case · 7.2 Component · 7.3 Domain class model · 7.4 Sequence: scenario evaluation

7.5 Sequence: near-real-time replay · 7.6 Sequence: model release · 7.7 Activity: pipeline

7.8 State machine: stress severity · 7.9 State machine: model lifecycle · 7.10 Deployment

8 Data Architecture

9 Machine Learning Architecture

10 Security, Governance and Compliance

11 Non-Functional Requirements and Their Realisation

12 Technology Stack

13 Risks, Limitations and Future Work

Appendix A Repository Map · Appendix B Service Interface Reference

3. Introduction

This document describes the architecture and detailed design of the Dynamic Resilient Grid (DRG) framework: a demand-side analytics system that forecasts neighbourhood-level electricity demand, detects statistically defined stress periods, and quantifies how electric-vehicle and heat-pump adoption amplifies peak demand on UK urban distribution networks.

3.1 Purpose

The purpose of this document is to define an implementable, reviewable architecture that satisfies the business requirements set out in DRG-BRD-001. It records the structure of the system, the responsibilities of each component, the interactions between them, the design decisions taken and the reasoning behind those decisions, so that the system can be built, assured, operated and extended by parties other than its original author.

3.2 Intended audience

Table 3.5 - Audience and the sections most relevant to each

Reader	Primary interest	Sections
Project sponsor / supervisor	That the design addresses the stated problem and scope	1-3, 13
Network planning stakeholders	The stress definition, scenario assumptions and what the outputs mean	2, 5, 7.4, 9
Software and data engineers	Component decomposition, interfaces, data contracts	6-8, App. A-B
Machine learning engineers	Feature design, validation protocol, model lifecycle and release control	9, 7.6, 7.9
Cloud platform and security reviewers	Deployment topology, identity model, data governance	7.10, 10, 11

3.3 How to read this document

Sections 1-5 establish *what* the system is for and where its boundaries lie. Sections 6-7 present the architecture, first in prose and then formally in UML. Sections 8-12 describe the data, machine-learning, security and technology dimensions of that architecture. Section 13 states, without softening, what the system does not do and where its results should not be relied upon. Readers who need only the operational picture may read Sections 5, 7.2, 7.10 and 12.

Notation

Diagrams follow UML 2.5 notation. Where a diagram departs from strict UML for legibility - for example, annotating a component diagram with the concrete Python package that realises each component - the departure is stated in the caption. Requirement identifiers in the form **FR-04** or **NFR-02** refer to DRG-BRD-001.

4. Project Overview

4.1 Problem statement

Urban low-voltage distribution networks in the United Kingdom were dimensioned for a domestic load profile that is now changing faster than the assets serving it. Two electrification vectors dominate that change. Uncontrolled domestic electric-vehicle charging concentrates a 7 kW load into the evening window that already contains the daily demand peak. Electrified heating replaces a gas load with an electrical one that rises precisely when the weather is coldest and occupancy highest.

Both loads are individually well understood. What network planners lack is a way to quantify, at the granularity of an actual neighbourhood and an actual half-hour, how far a given adoption trajectory pushes local demand beyond its historical envelope - and to do so from the data they already hold, namely smart-meter consumption records, without waiting for network telemetry that may never be instrumented at that level.

Conventional practice relies on historical averages and diversified design figures. These are sound for steady-state planning but silent on three questions that matter for electrification: how *often* demand will exceed its historical envelope, for how *long* each such episode will last, and which conditions *drive* those episodes.

4.2 Aim

Aim

To design, implement and evaluate an AI-driven forecasting and electrification stress-modelling framework that quantifies neighbourhood-level demand risk under future electric-vehicle and heat-pump adoption scenarios, using demand-side smart-meter data and publicly available contextual data only.

4.3 Objectives

Table 4.6 - Project objectives and their realisation in the architecture

Ref	Objective	Realised by
OBJ-1	Analyse neighbourhood-level consumption patterns from UK smart-meter data	Ingestion and EDA components (Sections 6.2, 8)
OBJ-2	Develop and validate machine-learning models for short-term demand forecasting	Forecasting component and validation protocol (Section 9)
OBJ-3	Implement statistical stress detection using percentile-based thresholds	Stress detection component (Sections 6.2, 7.8)
OBJ-4	Simulate EV and heat-pump adoption scenarios and quantify peak amplification	Electrification simulation component (Sections 7.3, 7.4)
OBJ-5	Conduct sensitivity analysis of stress frequency and duration	Sensitivity analysis component (Section 7.4)
OBJ-6	Apply explainable AI to interpret model outputs	SHAP explainability component (Section 9.5)
OBJ-7	Deliver a dashboard visualising forecasts, alerts and scenario comparisons	Presentation layer and streaming replay (Sections 6.2, 7.5)

4.4 Research questions

1. **RQ1** - How accurately can machine-learning models forecast half-hourly local electricity demand using smart-meter data?
2. **RQ2** - How sensitive is neighbourhood peak demand to varying EV and heat-pump adoption levels?
3. **RQ3** - To what extent does electrification amplify stress frequency and peak intensity?
4. **RQ4** - Which temporal and statistical features contribute most to high-demand stress prediction?

Each question maps onto a measurable output of the system: RQ1 to the evaluation metrics of the model ladder; RQ2 and RQ3 to the adoption sensitivity grid; RQ4 to the stress-conditional SHAP attribution. The architecture is arranged so that answering them is a by-product of running the pipeline, not a separate manual analysis.

4.5 What the system is not

Boundary of validity

DRG performs **no physical power-flow modelling**. It has no network topology, no transformer or cable ratings, and no voltage or thermal model. "Stress" is a statistical statement about a demand distribution, not an engineering statement about an asset. The output identifies *where and when to look* and *how much worse electrification makes it*; it does not by itself justify a reinforcement decision, which requires asset data the source dataset does not contain.

5. Scope

5.1 In scope

Table 5.7 - Functional scope of the delivered system

Ref	Capability	Description
SC-01	Data ingestion	Memory-bounded, chunked ingestion of the Low Carbon London half-hourly release; cleaning, outlier treatment, gap repair, aggregation of households into fixed-size neighbourhood groups.
SC-02	Contextual data acquisition	Retrieval of observed temperature and grid carbon intensity from public APIs, with caching, retry and documented degradation when unavailable.
SC-03	Feature engineering	Calendar, cyclical, lag, rolling-statistic, ramp and exogenous features, computed identically for training and serving.
SC-04	Demand forecasting	A ladder of models - seasonal naive, linear/ridge, gradient boosting, deep sequence - evaluated on a common held-out chronological window.
SC-05	Statistical stress detection	Percentile and standard-deviation thresholds; stress event extraction with frequency, duration, intensity and seasonal decomposition.
SC-06	Electrification simulation	Bottom-up, household-level EV charging and heat-pump load models with diversity and Monte-Carlo repetition.
SC-07	Sensitivity analysis	Full adoption grid, peak amplification, stress escalation, adoption elasticity and seasonal split.
SC-08	Explainability	Global and stress-conditional SHAP attribution for the champion model.
SC-09	Near-real-time prototype	Replay of historical demand as a simulated live feed with rolling forecasts and pre-emptive stress alerts.
SC-10	Decision-support interfaces	A REST service and an interactive dashboard exposing forecasts, stress, scenarios and explanations.
SC-11	Cloud deployment	Reproducible Azure infrastructure as code, containerised services, scheduled retraining.
SC-12	Model lifecycle control	Automated promotion gating and blue/green model release with rollback.

5.2 Out of scope

Table 5.8 - Explicit exclusions and the reason for each

Ref	Excluded	Reason
OS-01	Power-flow, voltage or thermal modelling	Requires network topology and asset ratings absent from the source dataset.
OS-02	Asset-level reinforcement recommendations	Follows from OS-01; DRG informs such decisions but cannot make them.
OS-03	Real-time control or switching actions	DRG is an analytical and advisory system with no operational technology interface.
OS-04	Smart-tariff and managed-charging optimisation	Deliberately excluded so the results characterise the <i>uncontrolled</i> case, which is the planning worst case. Identified as future work (Section 13).
OS-05	Individual household profiling or targeting	Prohibited by the data licence and unnecessary for the analytical purpose.
OS-06	Generation, storage and export modelling	The source data measures import only.
OS-07	Geographic or GIS-based feeder mapping	Feeder identifiers are not present in the anonymised dataset.

5.3 Assumptions

Table 5.9 - Assumptions on which the design depends

Ref	Assumption	If invalid
AS-01	Fixed-size hash-derived household groups are an acceptable proxy for LV feeder groups for statistical purposes.	Absolute stress figures remain valid per group but cannot be mapped to a physical asset.
AS-02	Historical consumption behaviour, absent electrification, remains broadly stationary over the planning horizon.	Baseline thresholds drift; retraining cadence must increase.
AS-03	EV charging is uncontrolled and predominantly begins in the evening window.	Peak amplification is overstated; managed charging would flatten it materially.
AS-04	Heat-pump electrical input scales with heating degrees and saturates at the unit rating.	Winter results shift; the model is parameterised so the relationship can be recalibrated.
AS-05	Public contextual APIs remain available under their current terms.	The system degrades to documented fallbacks without failing.

5.4 Constraints

Table 5.10 - Constraints imposed on the design

Ref	Constraint	Architectural consequence
CN-01	Data licence	Raw smart-meter data may not be redistributed or published. Ingestion must run where the data already resides; the repository must remain runnable without it.
CN-02	Commodity hardware	The pipeline must complete on a machine with 8 GB RAM, forcing chunked ingestion and immediate aggregation rather than whole-dataset loading.
CN-03	No asset data	Stress must be defined statistically rather than against a rating.
CN-04	Cost ceiling	Cloud design favours scale-to-zero serverless hosting over always-on clusters.
CN-05	Reproducibility	Every stochastic component must be seeded and every run must emit a machine-readable report.

6. Stakeholders and System Actors

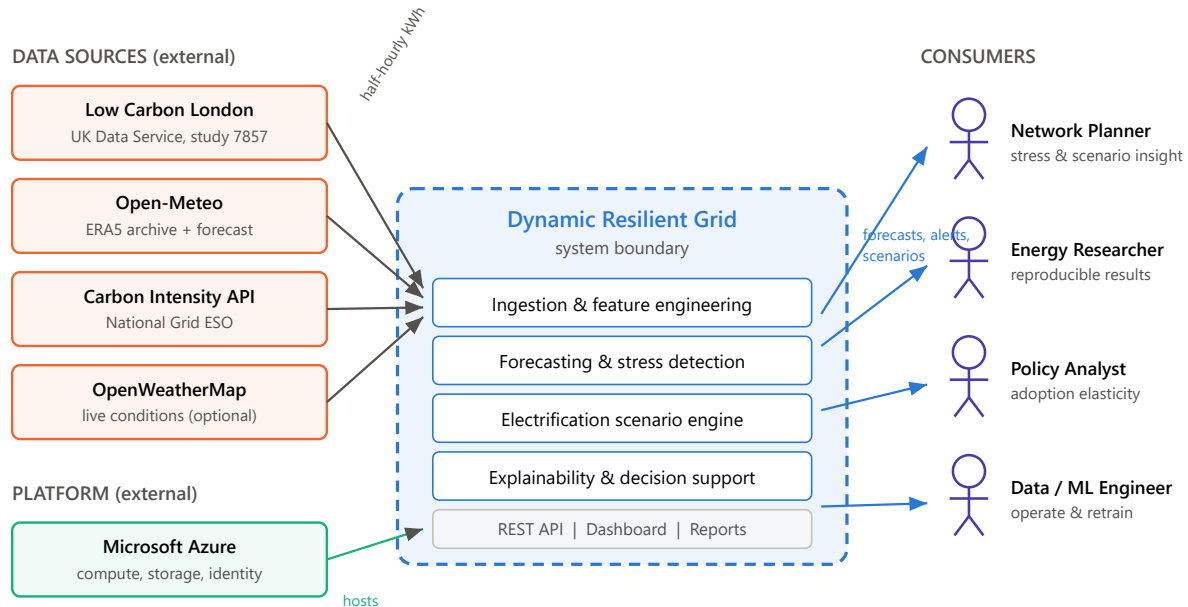
The distinction matters for the use-case model in Section 7.1: a *stakeholder* has an interest in the system's outcomes; an *actor* interacts with the system directly.

Table 6.11 - Stakeholders, actors and their concerns

Party	Actor?	Principal concern
Network Planner (DNO)	Yes	Which neighbourhoods approach their demand envelope, how electrification changes that, and how much headroom remains.
Urban Energy Researcher	Yes	Reproducible method, defensible parameters, access to underlying results.
Policy & Strategy Analyst	Yes	Adoption elasticity - the marginal effect of each additional tranche of EV or heat-pump uptake.
Data Engineer	Yes	Ingestion of the licensed extract, data quality, pipeline operation.
ML Engineer	Yes	Model quality, retraining, safe release to the serving endpoint.
Scheduler (time-triggered)	Yes	Non-human actor initiating the nightly pipeline and weekly retraining.
Energy Supplier	No	Flexibility-product design informed by stress timing.
Data Protection Officer	No	Licence compliance and absence of personal data.
Project Sponsor	No	Delivery against the business case.

7. System Context

The context diagram fixes the system boundary: what DRG owns, what it consumes, and who consumes it. Everything inside the dashed boundary is delivered by this project; everything outside is a dependency whose availability and terms are outside the project's control.



Data crossing the boundary inbound is anonymised half-hourly consumption and public contextual data.
Data crossing outbound is aggregated analytics only: no household-level record ever leaves the system.

Figure 7.1 - System context. The boundary is drawn at the analytics system; the licensed dataset, the public APIs and the cloud platform are all external dependencies.

Table 7.12 - External interfaces

Interface	Direction	Protocol	Failure behaviour
Low Carbon London extract	Inbound	File (CSV)	Absent extract triggers the calibrated synthetic generator; provenance recorded in the data.
Open-Meteo archive	Inbound	HTTPS / JSON	Retry with backoff, then deterministic synthetic temperature; row tagged <code>synthetic</code> .
Carbon Intensity API	Inbound	HTTPS / JSON	Archive begins 2018; earlier periods use a documented climatology projection tagged <code>climatology-proxy</code> .
OpenWeatherMap	Inbound	HTTPS / JSON	Used only when a key is configured; otherwise Open-Meteo serves live conditions.
DRG REST API	Outbound	HTTPS / JSON	Returns 503 with the remediation command when a required artifact is missing.
DRG Dashboard	Outbound	HTTPS / HTML	Renders an actionable message rather than a stack trace when data is unavailable.

8. Architecture Overview

8.1 Architectural drivers

Four forces shaped the architecture more than any others.

1. **The data cannot leave its custody, and may not be redistributed.** This drives a design in which the repository is fully runnable *without* the licensed data, and ingestion is a pluggable stage rather than a precondition for every other stage.
2. **Memory is bounded before the dataset is.** Ingestion must aggregate inside the read loop, not after it.
3. **An analytical claim is only as good as its reproducibility.** Every stochastic element is seeded, every stage caches a typed artifact, and every run emits a machine-readable report that regenerates the published figures.
4. **Scenario results are policy-adjacent.** Parameters that materially change the headline number must be explicit, defensible against published planning figures, and auditable in the output rather than buried in code.

8.2 Layered view

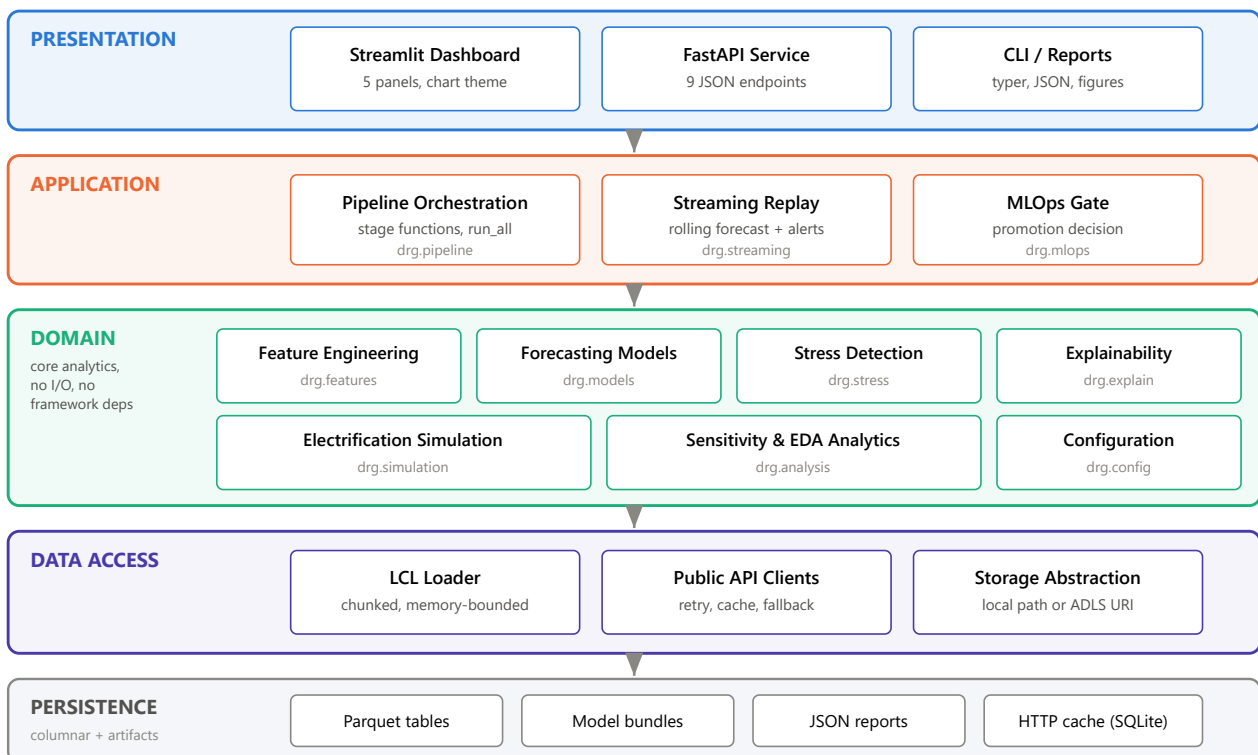


Figure 8.2 - Layered architecture. Dependencies point downward only. The domain layer carries the analytics and has no dependency on any web framework, cloud SDK or I/O concern, which is what allows it to be unit-tested offline in seconds.

8.3 Design principles

Table 8.13 - Principles applied throughout the design

Principle	Application in DRG
Stages are pure functions over tables	Each stage reads a typed table, returns a typed table and caches it. Any stage can be re-run alone; the cloud training job and the local CLI call the identical functions, so there is no second code path to keep in step.
One feature builder	Training, replay and the API all call <code>drg.features.engineering</code> . Train/serve skew becomes structurally impossible rather than a discipline to maintain.
Never claim data you do not have	Every derived series carries a <code>source</code> column and degrades in documented steps: measured, then proxy, then synthetic.
Fix the yardstick before the experiment	Stress thresholds are estimated on the historical baseline and then held constant, so a scenario that raises demand raises measured stress instead of moving the threshold with it.
Assert physical sanity	The simulation reports ADMD per technology and energy per vehicle per day; automated tests assert equipment ratings are never exceeded and that daily EV energy stays within the range implied by UK average mileage.
Fail loudly, degrade quietly	A missing model is a 503 naming the command that fixes it; an unavailable API is a logged fallback that does not stop the pipeline.

8.4 Key architectural decisions

Table 8.14 - Decision record

Ref	Decision	Rationale and rejected alternative
AD-01	Statistical stress definition (P95 of own history)	No asset ratings exist in the dataset. <i>Rejected</i> : assuming a nominal transformer rating, which would have produced authoritative-looking numbers resting on an invented constant.
AD-02	Gradient boosting as the primary model	The target responds to sharp threshold-like interactions (evening peak x cold x weekday) that trees capture directly, and TreeSHAP gives exact attribution cheaply. <i>Rejected</i> : deep sequence model as primary - marginally worse here and far harder to explain and serve.
AD-03	Bottom-up, household-level electrification simulation	Produces a genuine diversity effect and exposes auditable per-household quantities (ADMD). <i>Rejected</i> : a flat multiplier on the aggregate profile, which cannot represent coincidence.
AD-04	Calibrated synthetic fallback dataset	Makes the entire framework runnable, testable and shareable without the licensed extract. <i>Rejected</i> : committing a data sample, which the licence forbids.
AD-05	Azure Container Apps over AKS	Three small services and a nightly batch job; scale-to-zero and a native cron job type cover the workload without a cluster to operate. <i>Rejected</i> : AKS, disproportionate to the load.
AD-06	Shared Azure Files volume between batch and serving	The batch job writes data and models that the API and dashboard read immediately; removes a separate synchronisation step. <i>Rejected</i> : blob-download-on-start, which adds cold-start latency and a failure mode.
AD-07	Separate application and model release pipelines	Code and models have different risk profiles, cadences and approvers. A model change must pass a quality gate; a documentation change must not.
AD-08	Tabular model always served, even if a sequence model scores better	The serving contract is a design matrix; sequence models need a frame. The trainer records the fact in its log rather than silently serving something the API cannot call.

9. UML Model

This section presents the system formally. Each diagram is followed by the design commentary that the notation alone cannot carry - in particular the reasoning behind the interactions, which is where most of the engineering judgement in this system resides.

9.1 Use case diagram

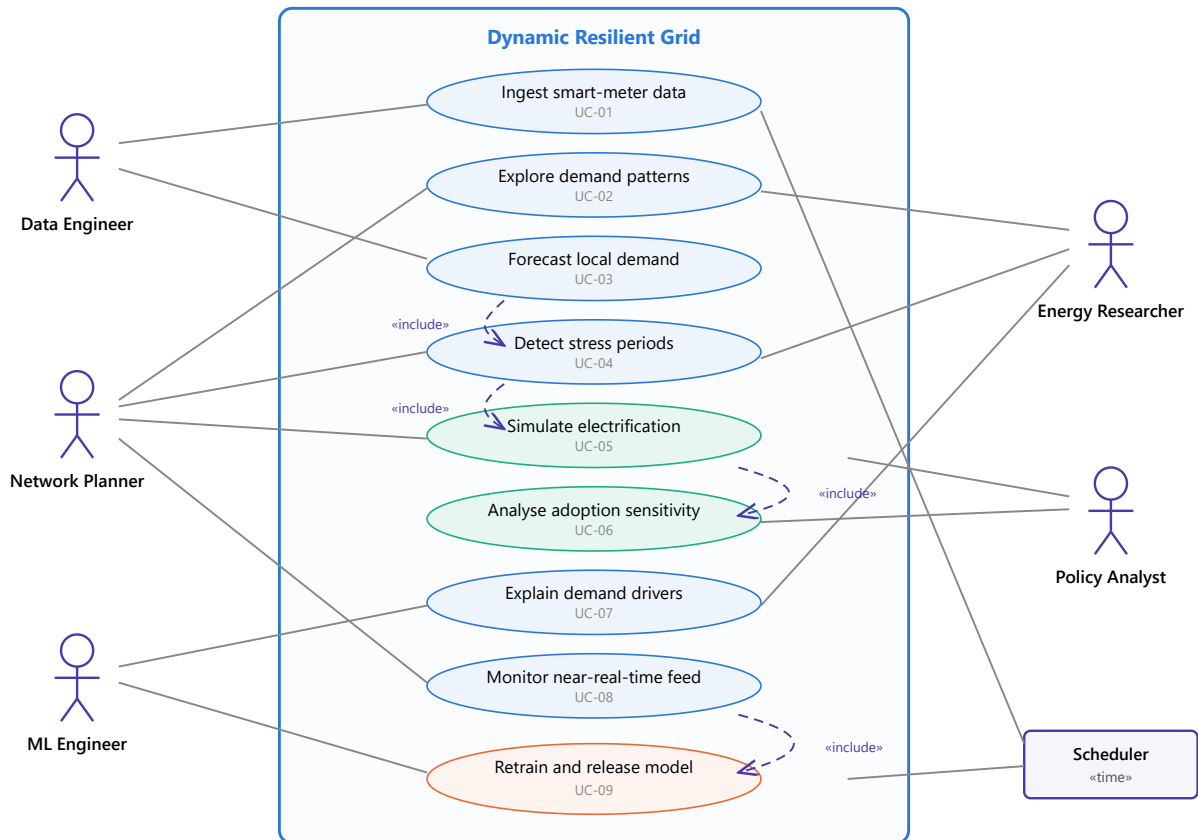


Figure 9.3 - Use case diagram. Green use cases are the analytical contribution that distinguishes DRG from a conventional forecasting study; the orange use case is the operational lifecycle that keeps the deployed model trustworthy.

Table 9.15 - Use case specifications (abridged)

Ref	Use case	Primary actor	Main flow and outcome
UC-01	Ingest smart-meter data	Data Engineer	Read the licensed extract in chunks, clean, aggregate to neighbourhoods, acquire contextual series. Outcome: a cleaned half-hourly table with recorded provenance.
UC-02	Explore demand patterns	Network Planner	Produce load curves, seasonal profiles, temperature response and peak-timing evidence. Outcome: EDA figures and a numeric summary.
UC-03	Forecast local demand	Network Planner	Train the model ladder, evaluate on a held-out chronological window, select and persist a champion. Outcome: forecasts with quantified accuracy.
UC-04	Detect stress periods	Network Planner	Apply percentile thresholds, extract contiguous events, summarise frequency, duration and intensity. <i>Includes</i> UC-03 when applied to forecasts.
UC-05	Simulate electrification	Network Planner	Add bottom-up EV and heat-pump load at a chosen adoption level and recompute stress against the fixed baseline thresholds. <i>Includes</i> UC-04.
UC-06	Analyse adoption sensitivity	Policy Analyst	Sweep the adoption grid; report peak amplification, stress escalation and elasticity. <i>Includes</i> UC-05.
UC-07	Explain demand drivers	Energy Researcher	Compute SHAP attribution globally and restricted to stress periods. Outcome: ranked drivers answering RQ4.
UC-08	Monitor near-real-time feed	Network Planner	Replay history as a live feed with rolling forecasts, severity state and pre-emptive alerts.
UC-09	Retrain and release model	ML Engineer / Scheduler	Retrain, evaluate against the promotion gate, and release by blue/green deployment with rollback. <i>Includes</i> UC-03.

9.2 Component diagram

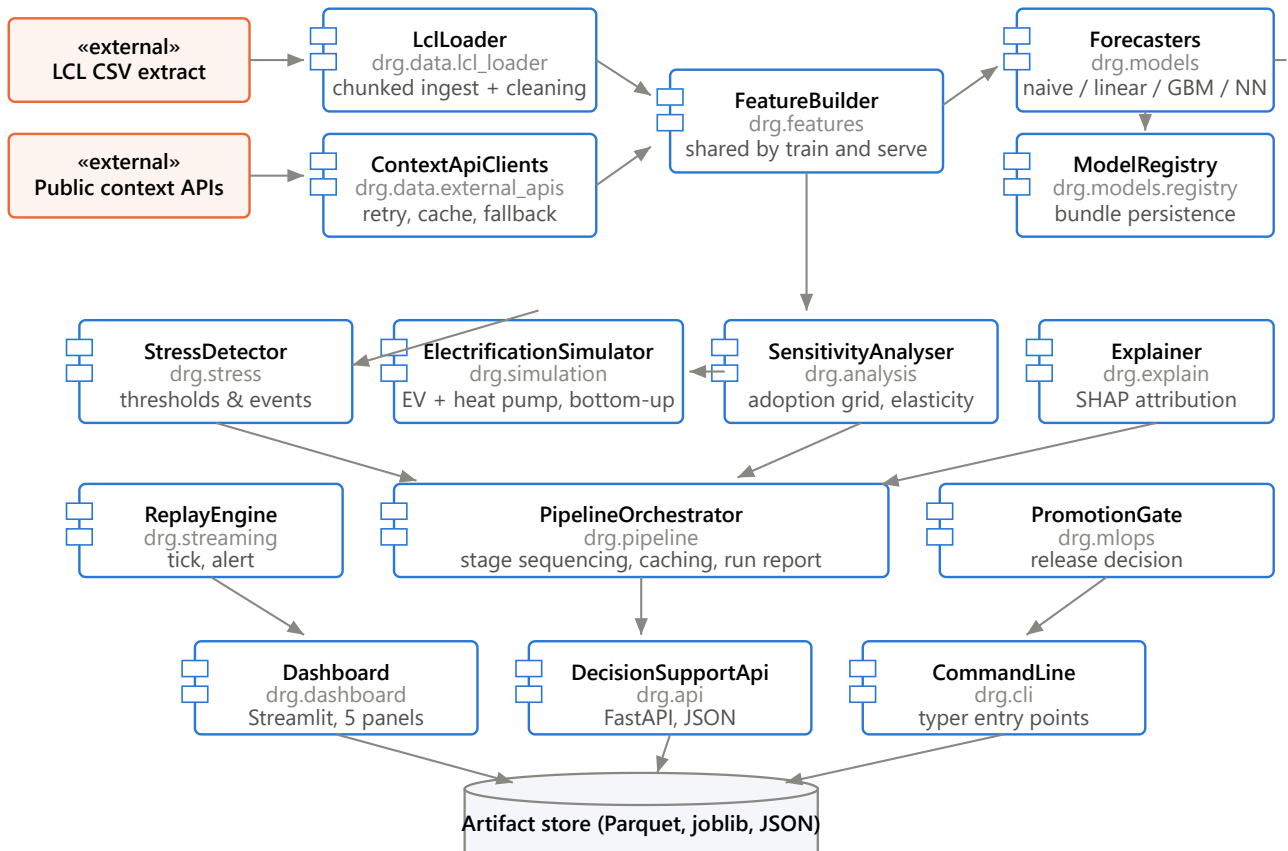


Figure 9.4 - Component diagram. Each component is annotated with the Python package that realises it (a deliberate departure from strict UML, for traceability between design and code). Arrows denote compile-time dependency.

Why the dependency direction matters

`ElectrificationSimulator` depends on nothing but numeric libraries, and `StressDetector` depends on nothing at all beyond the demand table. That is deliberate: these two components carry the analytical claims of the project, so they are kept free of configuration, I/O and framework coupling and can be exercised exhaustively in unit tests. The orchestrator, not the domain components, owns all knowledge of where data lives.

9.3 Domain class model

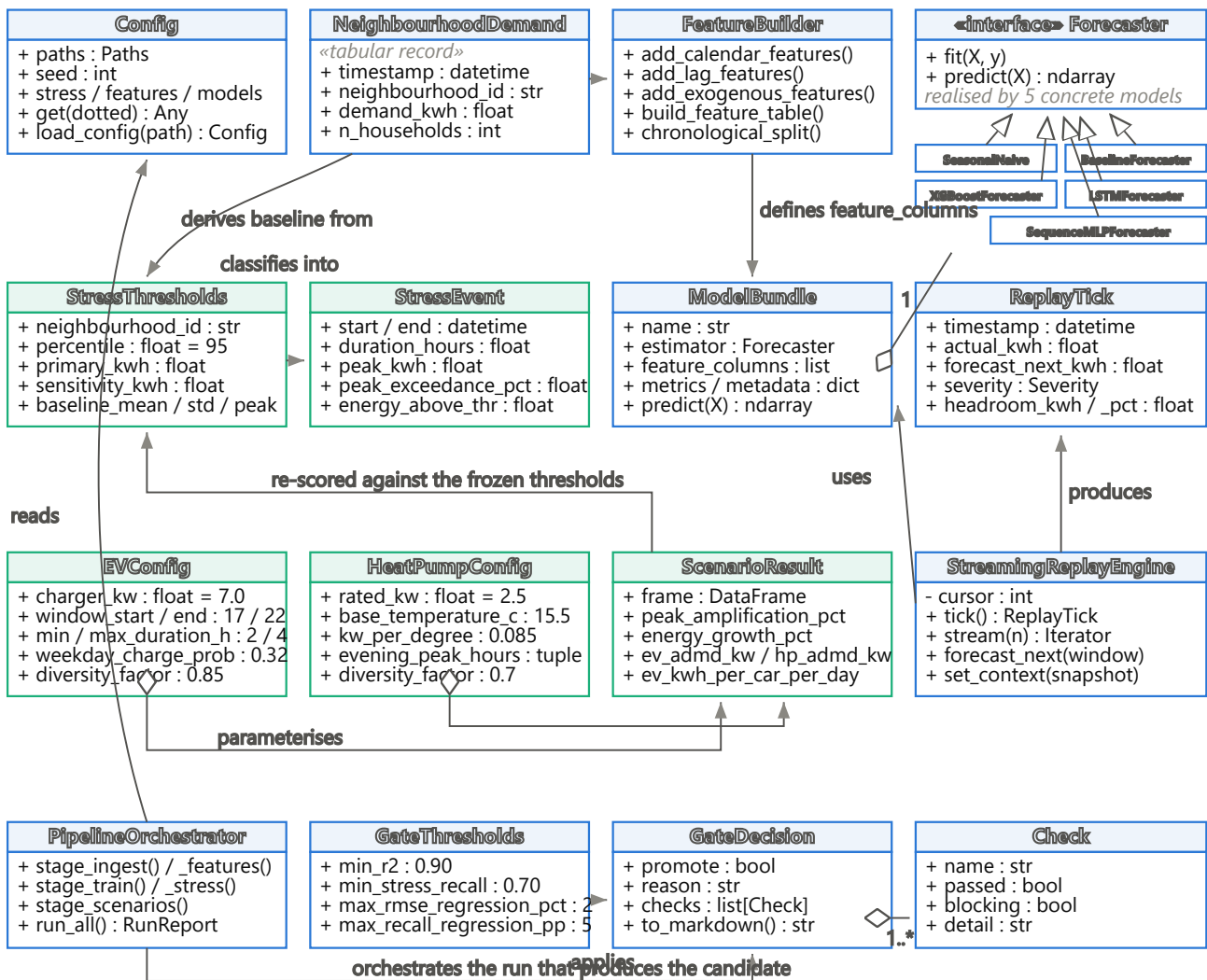


Figure 9.5 - Domain class model. Blue classes are infrastructure and modelling types; green classes carry the analytical domain concepts (stress and electrification) that constitute the project's contribution. Tabular records are shown as classes for clarity.

Configuration objects as first-class domain types

EVConfig and HeatPumpConfig are not bags of settings; they are the scientific parameters on which every headline number depends. Modelling them as typed classes with documented defaults, a `from_config()` factory and unit tests asserting their physical consequences is what turns "we assumed 7 kW charging" from a code comment into a reviewable artefact.

9.4 Sequence: electrification scenario evaluation

This is the interaction that answers RQ2 and RQ3. The critical design point is the order of operations: thresholds are resolved from the *historical baseline* before any simulated load is added, so the comparison is against a fixed yardstick.

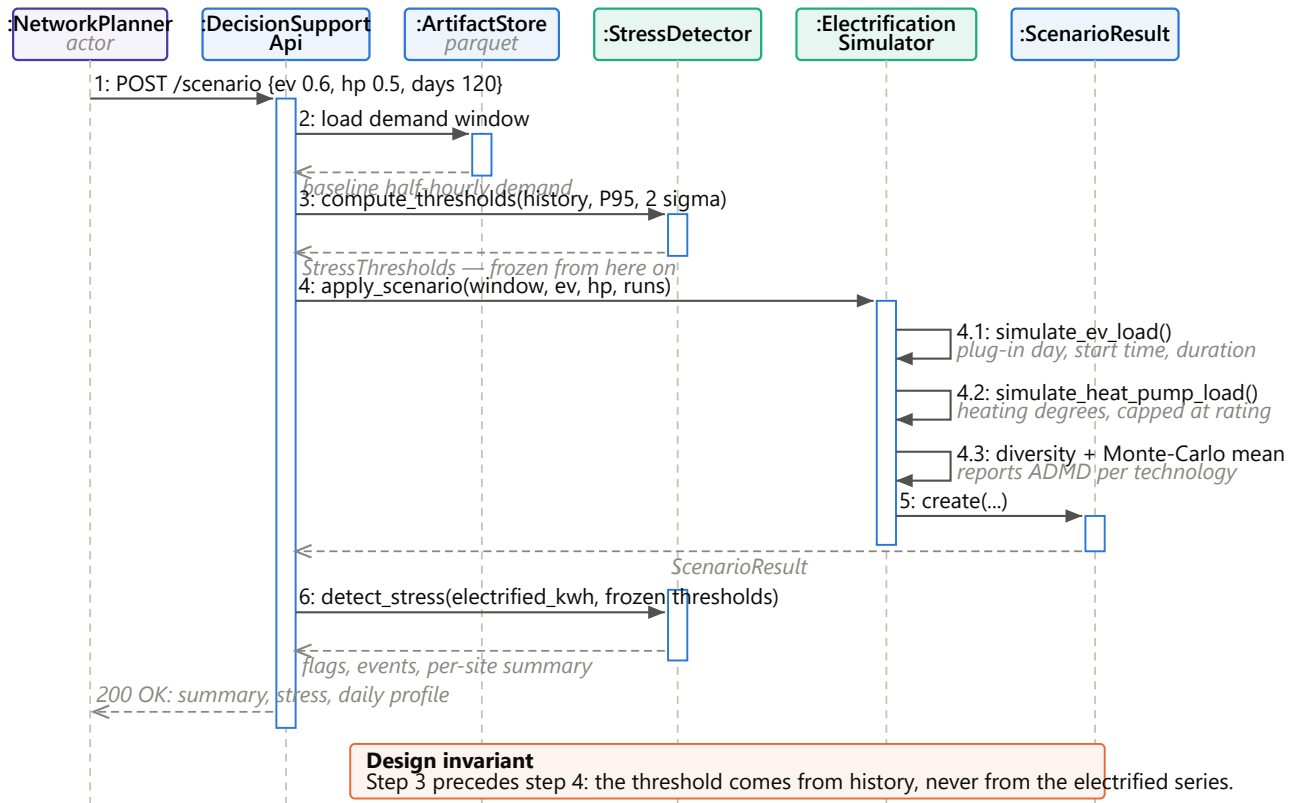


Figure 9.6 - Sequence diagram - on-demand electrification scenario evaluation via the REST service. The same interaction, minus the HTTP boundary, is executed by the dashboard scenario studio and by the batch sensitivity grid.

9.5 Sequence: near-real-time replay tick

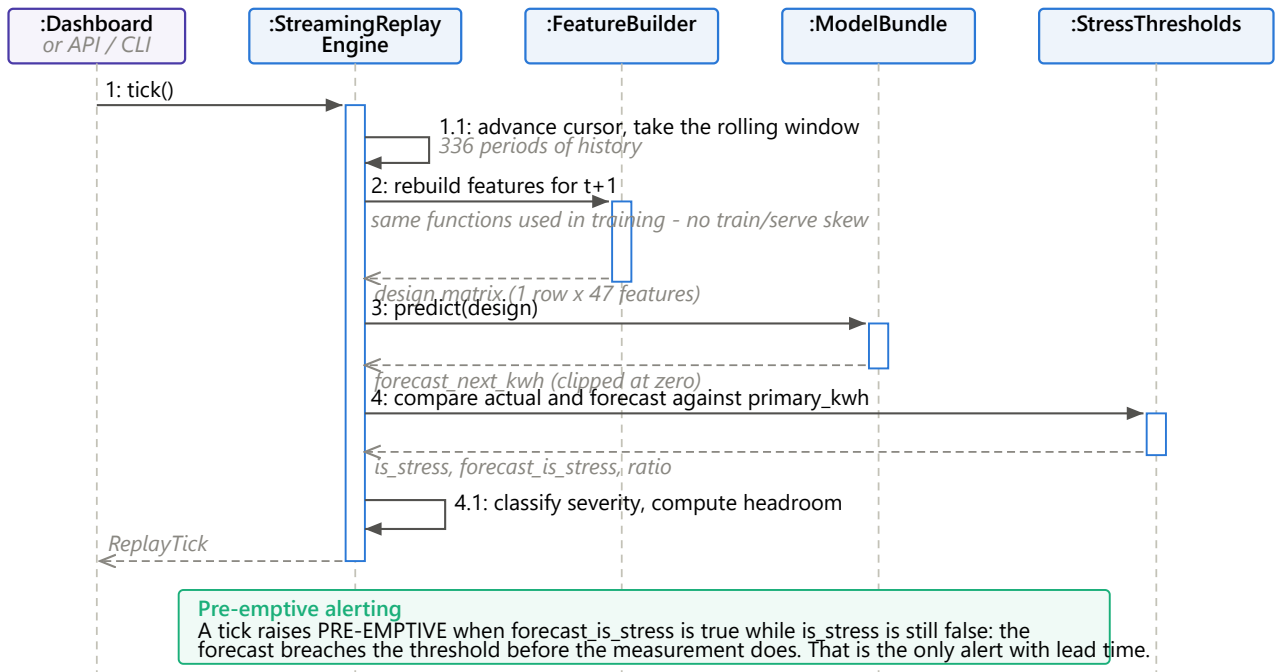


Figure 9.7 - Sequence diagram - one replay tick. The engine is pull-based and synchronous so the CLI demonstration, the dashboard panel and the REST endpoint share a single implementation.

9.6 Sequence: model retraining and release

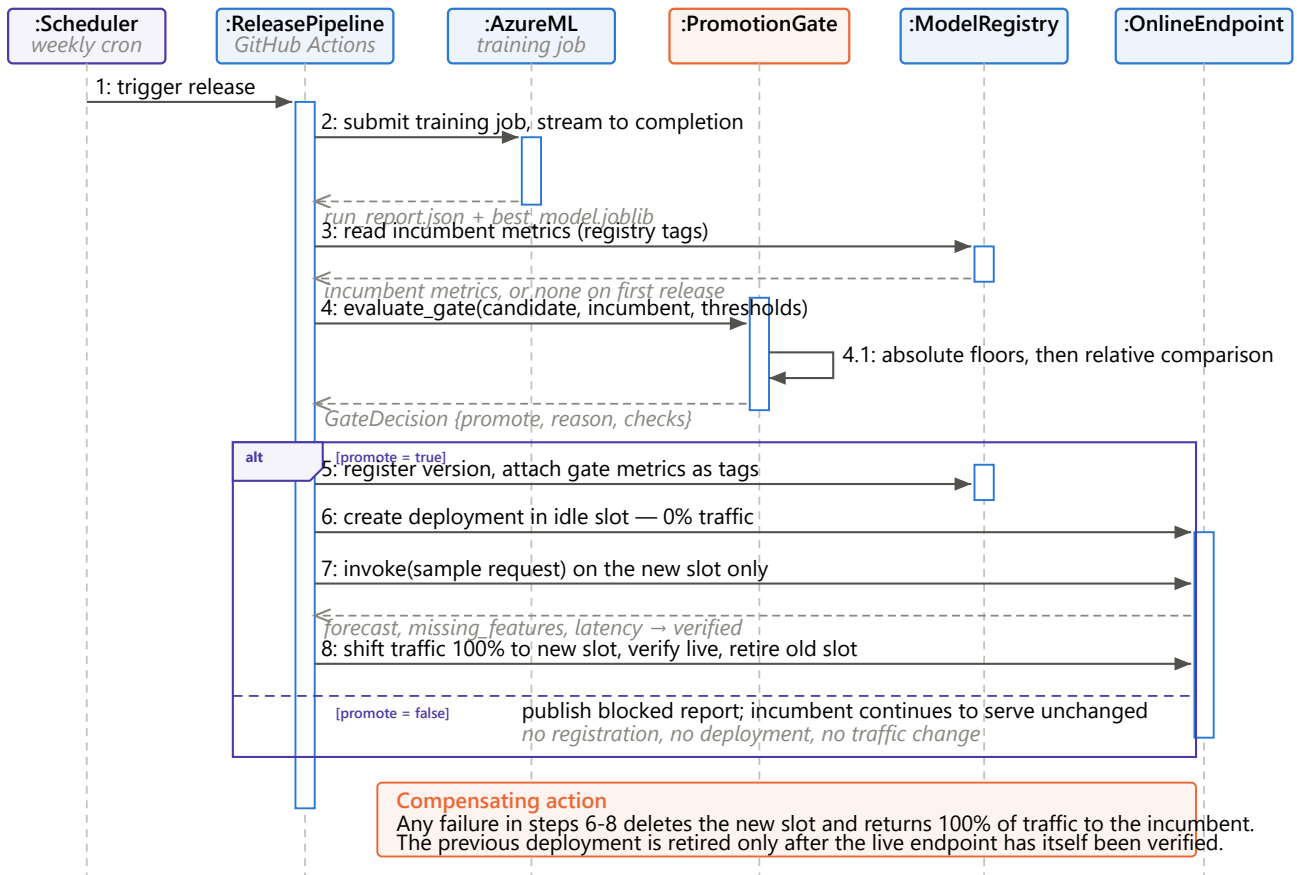


Figure 9.8 - Sequence diagram - automated model release. The `alt` fragment is the safety property: a model that fails the gate never reaches the registry, let alone the endpoint.

9.7 Activity: end-to-end analysis pipeline

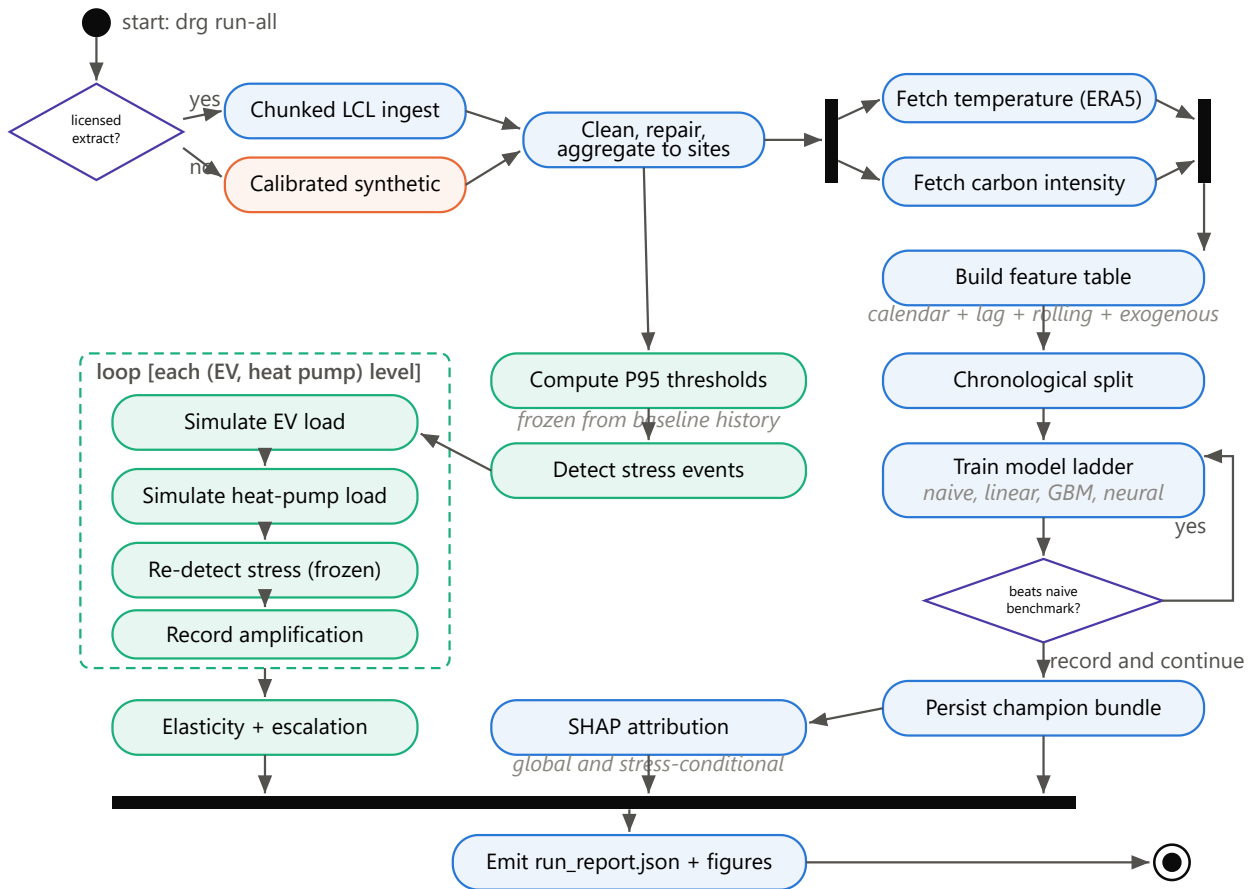
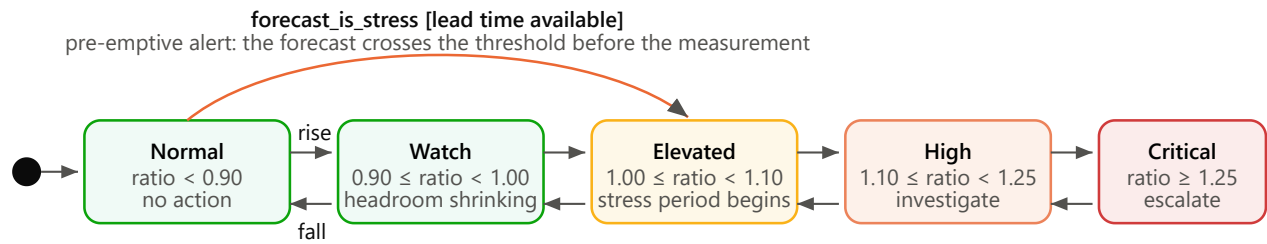


Figure 9.9 - Activity diagram - the `run-all` pipeline. The fork after cleaning shows that contextual data acquisition is parallelisable; the loop is the adoption grid; the join guarantees a single consolidated run report.

9.8 State machine: neighbourhood stress severity



Guard condition

ratio = demand / threshold_primary_kwh, where the threshold is the 95th percentile of that neighbourhood's own baseline history.

Event definition

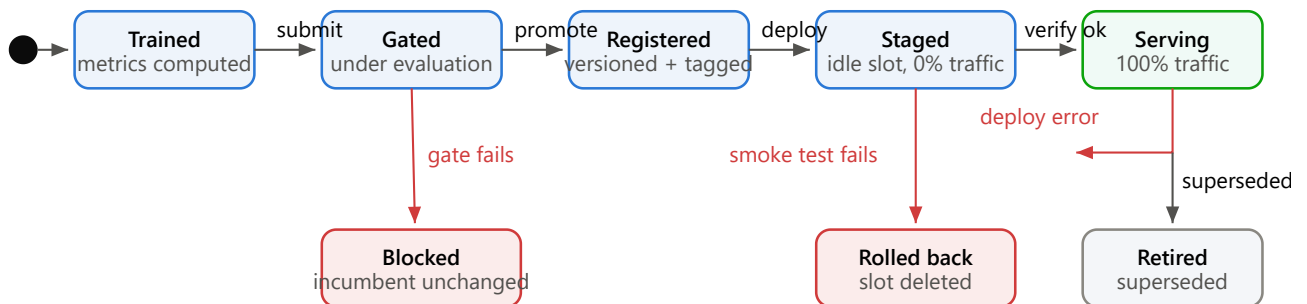
A contiguous run of two or more half-hours at Elevated or above constitutes one stress event, which is the unit reported in the duration statistics.

Presentation rule

Severity is never conveyed by colour alone: each state renders with an icon and its text label, so the state survives greyscale printing and colour-vision deficiency.

Figure 9.10 - State machine - stress severity of a neighbourhood at a given half-hour. The orange transition is the pre-emptive path that gives an operator lead time.

9.9 State machine: model lifecycle



Terminal states Blocked and Rolled back are both safe: the previously Serving model keeps 100% of traffic.

A model reaches Retired only after its successor has been verified while live: the incumbent is never deleted speculatively.

Figure 9.11 - State machine - lifecycle of a forecasting model from training to retirement. Red states are the failure paths; both preserve the currently serving model.

9.10 Deployment diagram

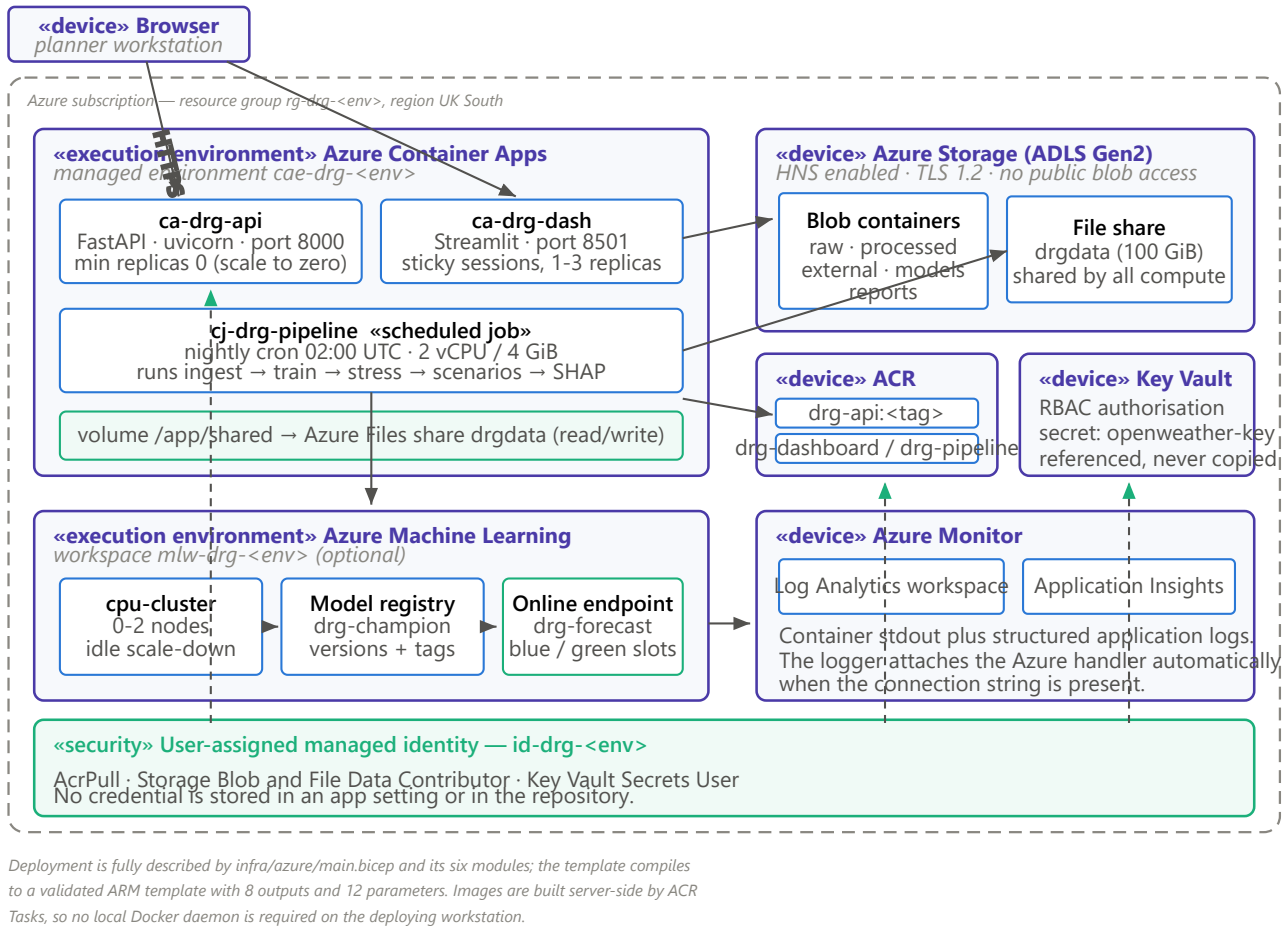


Figure 9.12 - Deployment diagram. The single shared Azure Files volume is the key topological choice: the batch job publishes data and models that the two services read immediately, removing a separate model-synchronisation step and its failure modes.

Table 9.16 - Deployment characteristics

Node	Scaling	Notes
ca-drg-api	0-5 replicas	HTTP-concurrency scale rule at 40 requests; liveness and readiness probes on <code>/health</code> . Scale-to-zero accepted in exchange for cold-start latency (see NFR-03).
ca-drg-dash	1-3 replicas	Minimum one replica and sticky sessions, because Streamlit holds per-session state.
cj-drg-pipeline	1 replica, cron	Nightly full pipeline; 2 vCPU / 4 GiB is sized from the observed 145 s local run with headroom for the full licensed extract.
AML cpu-cluster	0-2 nodes	Idles at zero nodes; used when the licensed extract exceeds what the container job can process comfortably.
drg-forecast endpoint	1 instance / slot	Two slots exist only transiently during a release.

10. Data Architecture

10.1 Data flow and lineage

Data moves through five persisted stages. Each is a typed columnar table, cached so that any stage can be re-run without repeating its predecessors, and each carries provenance forward.

Table 10.17 - Persisted data products

Artifact	Format	Content and grain
<code>neighbourhood_demand</code>	Parquet	One row per (timestamp, neighbourhood). Cleaned half-hourly demand, household count, temperature, provenance.
<code>weather_hourly</code>	Parquet	Observed temperature aligned to the study index, with source tag.
<code>carbon_intensity</code>	Parquet	Half-hourly gCO2/kWh with a per-row provenance tag: measured, climatology-proxy or synthetic.
<code>feature_table</code>	Parquet	Model-ready design matrix, 47 numeric features plus identifiers and target.
<code>best_model.joblib</code>	joblib + JSON	Champion bundle: estimator, feature column order, training window, metrics.
<code>run_report.json</code>	JSON	The single machine-readable record of a run: data provenance, EDA summary, model metrics, stress statistics, electrification headlines, SHAP drivers.

10.2 Ingestion design

The published Low Carbon London half-hourly release is several gigabytes of CSV with the columns `LCLid`, `stdorToU`, `DateTime`, `KWH/hh` (per half hour) . Three design responses follow from constraint CN-02.

- **Aggregate inside the read loop.** Each chunk is grouped to (timestamp, neighbourhood) before the next chunk is read, so peak memory is proportional to `chunk_size` , not to the file.
- **Stable grouping.** Households map to neighbourhoods by a BLAKE2b hash of the meter identifier modulo the group count. Unlike Python's built-in `hash()` , which is salted per process, this is reproducible across runs and machines.
- **Tolerant column resolution.** The publisher's column spellings vary between releases, so columns are matched case- and whitespace-insensitively with documented fallbacks.

10.3 Data quality treatment

Table 10.18 - Cleaning rules and their justification

Rule	Parameter	Justification
Grid regularisation	30-minute index	Reindexing to a complete grid makes missingness explicit rather than implicit.
Robust outlier clipping	6 x MAD	Median absolute deviation is not itself distorted by the spikes it is meant to detect, unlike a standard-deviation rule.
Short-gap interpolation	≤ 6 periods, interior only	Meter communication losses of up to three hours are recoverable; longer gaps are not invented.
Day coverage filter	$\geq 80\%$ of 48 periods	A day with heavy missingness would understate its own peak and corrupt the percentile threshold.

Provenance is a first-class column, not documentation

The carbon-intensity API archive begins in 2018, while the study period is 2011-2014. Rather than silently substituting or silently omitting, the client projects a month-by-half-hour climatology from a recent reference year and tags every row `climatology-proxy`. A reader of the data can therefore tell measurement from projection without consulting this document. The same discipline applies to temperature and to the demand table itself.

11. Machine Learning Architecture

11.1 Feature design

Table 11.19 - Feature groups

Group	Features	Rationale
Calendar	hour, half-hour period (0-47), day of week, month, season, week of year, weekend flag, UK bank-holiday flag, evening-peak flag	Domestic demand is driven by routine before it is driven by weather.
Cyclical	sin/cos of period, day of week and day of year	Gives linear models the same access to periodicity that trees obtain by splitting.
Lags	t-1, t-2, t-3, t-48 (yesterday), t-96, t-336 (last week)	Autoregressive structure; the weekly lag proves to be the dominant driver.
Rolling statistics	mean, standard deviation, min, max over 3 h, 1 day and 1 week	Local level and volatility, all computed on shifted values.
Dynamics	ramp rate, ramp acceleration, daily and weekly deltas, load factor	The rate of change discriminates the rising edge of the evening peak.
Exogenous	temperature, heating and cooling degrees, lagged and smoothed temperature, HDD x evening interaction, carbon intensity	Weather sensitivity is the channel through which heat-pump electrification will act.

Two invariants, both enforced by automated tests

Rolling statistics are computed from `shift(1)` values, so the current observation can never enter its own prediction; and every lag is computed within a neighbourhood group, so no information crosses site boundaries. These are the two ways a load-forecasting result is most commonly and most quietly invalidated.

11.2 Validation protocol

Splitting is chronological throughout: a training window, then a validation window used for early stopping, then a held-out test window of 60 days that no model sees during fitting. The champion is additionally subjected to rolling-origin cross-validation over five expanding windows, so that a favourable test period cannot masquerade as model quality.

11.3 Evaluation metrics

Accuracy metrics alone are insufficient for a stress application, so the evaluation reports three families.

Table 11.20 - Metric families and why each is present

Family	Metrics	Purpose
Accuracy	MAE, RMSE, MAPE, sMAPE, R^2 , bias, P95 absolute error	Answers RQ1 and satisfies the proposal's stated metric set.
Peak-restricted accuracy	MAE, RMSE, MAPE over the 17:00-21:30 window	Average accuracy can hide poor performance exactly where the application needs it.
Stress-alarm skill	Precision, recall, F1 against the P95 threshold	Treats the forecast as the alarm it will actually be used as.

11.4 Model selection and serving

The ladder exists so the headline number carries meaning: a 3.37% MAPE is informative only against the 7.66% that a seasonal-naive rule achieves on the same window. Gradient boosting was selected as the primary model (AD-02) because the target responds to sharp, threshold-like interactions and because TreeSHAP provides exact attribution at negligible cost.

One serving constraint is deliberate and visible in the code: the model actually deployed is always a tabular estimator, because the serving contract is a design matrix. If a sequence model scores better, the trainer logs that fact explicitly and still serves the tabular champion, rather than silently persisting an artefact the API cannot call.

11.5 Explainability

Two SHAP views are produced. The global view ranks drivers over a sample of the test window. The stress-conditional view repeats the analysis restricted to observations above the stress threshold - which is the view a network planner needs, because it answers "what drives the half-hours I care about" rather than "what drives demand on average". A kernel explainer covers non-tree models.

11.6 Model lifecycle and release control

Retraining is scheduled; release is gated. The promotion gate applies absolute floors that a model must meet regardless of the incumbent, and relative comparisons against the currently deployed model. Two design choices in the gate are worth stating explicitly.

- **Stress recall is gated harder than precision.** A missed stress period is a missed reinforcement signal; a false alarm costs a glance at a dashboard. The asymmetry in the thresholds encodes that asymmetry in consequence.
- **A missing metric is a failure, not a pass.** The most dangerous gate is one that approves a model because a metric was absent.

Gate metrics are written onto the registered model as tags, so the next release finds its incumbent without a separate metrics store to keep synchronised.

12. Security, Governance and Compliance

12.1 Data governance

Table 12.21 - Governance controls

Obligation	Control implemented
UK Data Service End User Licence	Raw smart-meter data is never committed to source control (<code>data/raw/</code> is version-control ignored) and is never transmitted to a public service by this code. Ingestion is designed to run where the data already resides.
No personal data processing	The source extract is anonymised at issue; DRG aggregates further, to groups of roughly 120 households, before any analysis. No output is attributable to a household.
Redistribution	The repository is runnable without the licensed data through a calibrated synthetic generator that contains no real household record, which is what makes the work shareable and reviewable.
Data residency	All Azure resources are provisioned in a single UK region by default.
Retention	Blob soft-delete is enabled for seven days; log retention defaults to thirty days.

12.2 Identity and secrets

- A single **user-assigned managed identity** carries every workload permission: registry pull, storage data access and Key Vault secret retrieval. No service-principal credential is stored in an application setting or in the repository.
- CI/CD authenticates to Azure with **OIDC federated credentials**; the repository holds only a client, tenant and subscription identifier, none of which is a secret capable of authenticating on its own.
- The one API key the system may use (OpenWeatherMap) is held in Key Vault and referenced by the container app, never copied into configuration. When no key is supplied the Key Vault reference is not created at all, so the deployment does not carry a dangling secret reference.

12.3 Platform hardening

Table 12.22 - Applied platform controls

Control	Setting
Container user	All three images run as a non-root user (uid 10001).
Transport	HTTPS enforced on ingress; <code>allowInsecure: false</code> ; storage requires TLS 1.2 and HTTPS-only traffic.
Public data exposure	Blob public access disabled; anonymous registry pull disabled; registry admin user disabled.
Key Vault	RBAC authorisation rather than access policies; soft-delete enabled.
Health gating	Liveness and readiness probes prevent traffic reaching an unhealthy replica.
Release safety	Model releases are gated, staged at zero traffic, verified and reversible; deployment concurrency is configured never to cancel a run mid-traffic-shift.

Residual risk accepted

The storage account key is required by the Azure Files volume mount and is therefore present in the deployment template as a secure parameter. This is a platform constraint of Container Apps file mounts rather than a design choice. It is mitigated by keeping the key out of application settings, by restricting the account to a single resource group, and by soft-delete. Migrating the mount to an identity-based protocol should be revisited when the platform supports it.

13. Non-Functional Requirements and Their Realisation

Table 13.23 - Non-functional requirements, design response and observed evidence

Ref	Quality	Requirement	Design response and evidence
NFR-01	Performance (batch)	A full analysis run completes within one hour on commodity hardware.	Chunked ingest, cached stages, vectorised simulation. Observed: 145 s end to end on a four-neighbourhood, 26-month dataset.
NFR-02	Memory	Runs within 8 GB RAM.	Aggregation inside the ingest loop; columnar storage; no whole-dataset materialisation.
NFR-03	Latency (interactive)	Scenario evaluation returns within a few seconds; forecast endpoints respond promptly.	Vectorised simulation over a bounded window; artifacts cached behind an in-process cache. Cold start is accepted as the cost of scale-to-zero and is removable by setting minimum replicas to one.
NFR-04	Accuracy	The primary model materially outperforms a seasonal-naive benchmark.	Observed: MAPE 3.37% versus 7.66% ; RMSE 0.95 versus 2.08; stress-alarm F1 0.90 versus 0.85.
NFR-05	Reproducibility	Identical inputs produce identical outputs.	Every stochastic component is explicitly seeded, including a process-stable per-site seed derived by CRC32 rather than by the salted built-in hash. Asserted by test.
NFR-06	Reliability	No external dependency can stop a run.	Retry with exponential backoff, disk caching, and documented degradation for every external call; offline execution is a supported and tested mode.
NFR-07	Observability	Deployed components are diagnosable without access to the host.	Structured logging that attaches an Application Insights handler automatically when the connection string is present; container logs shipped to Log Analytics.
NFR-08	Testability	Core analytics are verifiable offline and quickly.	Domain layer free of I/O and framework dependencies. Observed: 103 automated tests in about 36 seconds , fully offline.
NFR-09	Maintainability	Consistent, statically checked code.	Enforced lint and format in continuous integration; typed configuration; a single documented feature builder.
NFR-10	Portability	Runs on Windows, Linux and in a container without code change.	Path abstraction, no shell dependencies in the library, three container images, and a documented fallback when platform-native binaries for the deep model are unavailable.
NFR-11	Accessibility	Visual outputs remain readable in greyscale and with colour-vision deficiency.	A validated categorical palette with checked separation, status conveyed by icon and label as well as colour, single y-axes, and a data table beside every chart.

Ref	Quality	Requirement	Design response and evidence
NFR-12	Cost	Cloud running cost remains modest for a research deployment.	Scale-to-zero API, serverless job scheduling, an ML cluster that idles at zero nodes; estimated USD 60-110 per month at development tier.

14. Technology Stack

Table 14.24 - Selected technologies and the reason for each

Layer	Technology	Reason for selection
Language	Python 3.10-3.12	The scientific and ML ecosystem the analysis depends on; matches the stated system requirements.
Data processing	pandas, NumPy, PyArrow	Columnar Parquet keeps intermediate tables compact and typed; chunked CSV reading is native.
Modelling	scikit-learn, XGBoost, PyTorch (optional)	Baselines, the gradient-boosted primary model and an optional deep sequence extension.
Explainability	SHAP (TreeSHAP)	Exact attribution for tree ensembles at low cost; kernel fallback for other model families.
Service	FastAPI, Uvicorn, Pydantic	Typed request validation and generated OpenAPI documentation with minimal code.
Interface	Streamlit, Plotly	Rapid delivery of an interactive analytical interface without a separate front-end build.
CLI	Typer, Rich	A discoverable command surface over the same stage functions the pipeline uses.
Configuration	YAML with environment overrides	One source of truth for scientific parameters; overridable per environment without code change.
Testing	pytest	Fast, offline, and expressive enough to assert physical properties as well as software behaviour.
Quality	ruff, black	Deterministic formatting and linting enforced in continuous integration.
Packaging	Docker (three images)	Separate API, dashboard and pipeline images so each scales and fails independently.
Infrastructure	Azure Bicep	Declarative, reviewable and idempotent; compiles to a validated ARM template.
Hosting	Azure Container Apps	Scale-to-zero services and a native scheduled job type without a cluster to operate.
ML platform	Azure ML (optional)	Managed training, MLflow tracking, versioned model registry and a blue/green online endpoint.
CI/CD	GitHub Actions with OIDC	Separate application and model pipelines; no long-lived credential in the repository.

15. Risks, Limitations and Future Work

15.1 Architectural risks

Table 15.25 - Risk register (architectural)

Ref	Risk	Impact	Mitigation
RA-01	Statistical stress is mistaken for an engineering assessment of asset capability	High	Stated in the aim, on the dashboard footer, in this document and in the repository README. The system reports exceedance of a demand distribution, never of a rating.
RA-02	Scenario parameters drive headline numbers and could be set to flatter a conclusion	High	Parameters are configuration, not code; defaults are justified against published planning figures; every scenario reports ADMD and energy per vehicle per day so a reviewer can audit them.
RA-03	Public API terms change or a service is withdrawn	Medium	Clients are isolated behind an interface with documented fallbacks; no pipeline stage fails on an unavailable API.
RA-04	Concept drift: historical thresholds become unrepresentative	Medium	Scheduled retraining, gated on stress recall against the incumbent, which surfaces degradation rather than absorbing it.
RA-05	Hash-derived neighbourhoods cannot be mapped to physical assets	Medium	Documented as assumption AS-01; the grouping function is a single injection point should feeder identifiers become available.
RA-06	Platform-native ML binaries unavailable on an analyst workstation	Low	Import guarded broadly; a scikit-learn sequence network substitutes automatically so the neural arm of the study remains executable.

15.2 Stated limitations

1. **No power-flow modelling.** Stress is statistical. Reinforcement decisions require asset ratings and topology that the dataset does not contain.
2. **Carbon intensity before 2018 is a projection, not a measurement.** It is tagged as such and contributes under one per cent of model attribution.
3. **Neighbourhood groups are hash-derived,** not geographic feeders.
4. **Only uncontrolled charging is modelled.** Smart tariffs and managed charging would flatten the reported peaks substantially; the reported figures are therefore an upper bound on unmanaged electrification, which is the appropriate planning case but not a forecast of what will occur.
5. **The reference results in this document were produced on the calibrated fallback dataset.** The pipeline, thresholds and simulation are unchanged when the licensed extract is supplied, but absolute figures will differ.

15.3 Future work

Table 15.26 - Prioritised extensions

Ref	Extension	Value
FW-01	Managed and smart-tariff charging profiles	Quantifies how much of the modelled peak amplification flexibility can remove - the natural policy question following from this work.
FW-02	Probabilistic forecasting (quantile or interval outputs)	Stress is a tail phenomenon; a predictive distribution serves it better than a point forecast.
FW-03	Feeder-level mapping where identifiers exist	Converts a statistical signal into an asset-addressable one.
FW-04	Solar PV and battery export modelling	Completes the local energy balance; requires export-capable metering data.
FW-05	Drift monitoring on live inputs	Detects covariate shift between scheduled retraining cycles.

16. Appendix A - Repository Map

Table 16.27 - Source layout and responsibilities

Path	Responsibility
configs/config.yaml	Single source of truth for every scientific and operational parameter.
src/drg/config.py	Typed configuration: YAML plus environment overrides, with resolved paths.
src/drg/data/	Chunked LCL loader, calibrated synthetic generator, public API clients.
src/drg/features/	The single feature builder shared by training and serving.
src/drg/models/	Baselines, gradient boosting, deep and torch-free neural models, evaluation, bundle registry, training orchestration.
src/drg/stress/	Threshold computation, flagging, event extraction, summaries and profiles.
src/drg/simulation/	Bottom-up EV and heat-pump models and scenario application.
src/drg/analysis/	Exploratory analysis figures; adoption sensitivity grid and elasticity.
src/drg/explain/	Global and stress-conditional SHAP attribution.
src/drg/streaming/	Pull-based replay engine with pre-emptive alerting.
src/drg/mlops/	Promotion gate: thresholds, decision logic and reporting.
src/drg/api/ , src/drg/dashboard/	REST service; Streamlit dashboard and its chart theme.
pipelines/	Azure ML entry point, online scoring script, promotion CLI, endpoint verifier, model spec generator.
infra/azure/ , infra/aml/	Bicep infrastructure modules; Azure ML environment, job, endpoint and deployment definitions.
docker/ , .github/workflows/	Three container images; continuous integration, application deployment and model deployment pipelines.
tests/	103 offline tests covering data, features, models, stress, simulation, streaming, API, dashboard, MLOps and workflow contracts.

17. Appendix B - Service Interface Reference

Table 17.28 - REST endpoints

Method	Path	Purpose and notable behaviour
GET	/health	Liveness and readiness, including whether data and model artifacts are present. Used by the container probes.
GET	/context	Live temperature and grid carbon intensity for the modelled location.
GET	/neighbourhoods	Configured sites with household counts, thresholds and data coverage.
GET	/forecast	Rolling one-step-ahead forecasts with severity, headroom and alert flags.
GET	/stress	Historical stress summary and recent events.
POST	/scenario	On-demand scenario evaluation. Adoption is bounded to [0, 1] and charger rating to 22 kW by request validation.
GET	/sensitivity	Cached adoption grid and headline amplification figures.
GET	/explain	SHAP driver ranking, globally and during stress.
GET	/metrics	Model evaluation metrics for the current champion.

Error contract

When a required artifact is absent the service returns HTTP 503 with the exact command that produces it, for example `Processed demand not found. Run: python -m drg.cli ingest`. An error message that tells the operator what to do next is part of the interface, not an afterthought.